Completely monotone and Bernstein functions in Lean 4: a kernel-checked library with a general subordination theorem, applied to planar universal optimality

Xavier Callens (SocrateAI Lab)

Preprint v1, 2026-10-25

doi:10.5281/zenodo.23653842 (not peer reviewed)

AI-assistance disclosure. This preprint, its Lean code, its scripts and its claim ledger were produced by AI agents (Claude, Anthropic) under the author's direction. The author is responsible for the content. Every Lean file was written and compiled by the producing AI agent. An independent AI audit by a separate agent has been done (`AUDIT.md`, verdict ACCEPT_WITH_CORRECTIONS; this version applies its corrections, see `REVISION.md`): all 11 Lean files and the 5 repository prerequisites (`Triangular.Riesz`, `Triangular.RieszFinite`, `Autoresearch.Yukawa`, `Autoresearch.KeldyshCM`, `Autoresearch.DipolarExciton`) were rebuilt from source with the producer's olean directory off `LEAN_PATH`; `Lean.collectAxioms` over all 167 library constants gave 0 outside the allowlist; the auditor's own planted `sorry`, axiom and transitive-`sorry` controls were caught and a clean control passed. The upstream openai/-math and Mathlib oleans were not rebuilt by the auditor and the upstream commit was not re-checked (the pinned copy has no `.git`). *There is still no human audit*, the 20 `LEDGER_UNAUDITED_TIER_A` flags remain open, and an AI audit does not close them. The paper-stage recompile of Section 4 (Tier X) was made by the producing pipeline and covers 3 of the 11 files.

Abstract

We build a small Lean 4 library of *completely monotone* (CM) and *Bernstein* functions on $(0,\infty)$ and use it to re-derive, and extend, the admissibility results that this repository needed for planar universal optimality (bilayer dipolar exciton, Yukawa, Rytova–Keldysh). **Tier A** (Lean 4.34.1, pinned Mathlib; axioms within {`propext`, `Classical.choice`, `Quot.sound`}; planted `sorry` and planted-axiom controls caught): 130 theorems, every one with a recorded `#print axioms`, in 10 modules. The CM definition is proved *equivalent* to upstream's `AdmissiblePotential`. We prove closure under positive combinations, products, translations and differences of translates; a locally uniform limit theorem whose limit's smoothness is derived; a Laplace-mixture theorem under a *weaker* integrability hypothesis than the earlier lemma; and the general *subordination theorem*: if f is CM and $\psi > 0$ is a Bernstein function then $f \circ \psi$ is CM, by a strong induction without the Faà di Bruno formula. Negative controls show that each of the listed hypotheses is needed. New consequences include a Yukawa-screened bilayer exciton, continuous Yukawa mixtures and the interaction of two aligned $+1, -2, +1$ quadrupoles; we also prove that the *field* of one such quadrupole on a test charge (its potential energy) is *not* CM for every layer spacing. **Honest accounting.** The library does *not* shorten the earlier proofs end to end: for H1, H2 and H3 together the measured dependency closure grows from 406 to 505 source lines; only the marginal cost of a new potential is small. Mathlib already contains the sign-free mirror notion (`AbsolutelyMonotoneOn`), so this is not a Mathlib-ready contribution. All triangular-lattice optimality corollaries are *conditional* on the upstream openai/math theorem, which is AI-generated and not peer reviewed. **Tier C:** the human demonstration and the quadrupole pair counting. **Tier X:** a 60-digit screen with controls, and the proof-length measurement. No rusty-SUNDIALS or LeanFlow change was made in this lane. We claim no breakthrough and no novelty; no literature search for novelty was made.

Tiers. [A]: kernel-checked Lean, axioms within the allowlist, planted controls caught. [L]: a cited source, quoted in the ledger (prose as extracted from the PDF, mathematics transcribed linearly). [C]: an argument

that is not machine-checked. [X]: floating-point or tool measurement, which never supports a claim. Identifiers such as `CM-A-0005` refer to `ledger/ledger.json` (Section 7). A Tier A tag attaches to the Lean statement, never to its physical gloss. Everything resting on upstream openai/math [1] is conditional on it.

1 Introduction and status

Cohn and Kumar [3] call a smooth f on an interval *completely monotonic* if $(-1)^k f^{(k)} \geq 0$ for all $k \geq 0$, and treat pair potentials as functions of the *squared* distance [L, CM-L-0002]. Upstream openai/math [1] proves, with an AI-generated Lean development, that the triangular lattice minimises the energy of every such potential among locally finite density-one planar configurations [A, CM-A-0001]; Petrache and Serfaty stated this planar Cohn–Kumar problem as open in 2019 [2] [L, CM-L-0003] and we did not re-check its later status. Each physical potential then needs one admissibility theorem. Three earlier preprints [12, 13, 14] gave such theorems with ad hoc inductions; the Riesz-energy corollaries and the textbook treatment of the upstream theorem are in [9, 10, 11]. Two sibling preprints of the same programme [15, 16] do not use them. This paper factors the ad hoc proofs into a library and measures what that buys.

What is and is not here. Done: the library (Section 4), its human proofs (Section 3) and the applications (Section 5). Not done, by design or for lack of time: the *hard* direction of the Bernstein–Widder theorem (every CM function is a Laplace transform), pointwise limits without smoothness, mixtures of an arbitrary CM family, Stieltjes mixtures and the Fourier criterion (a CM two-dimensional Fourier transform in k^2 gives CM in r^2, preregistered with prior 0.3); no Mathlib pull request, no Zulip post. The library is on the half-line $(0, \infty)$ only.

2 Setting

Listing 1: Definitions and the bridge to upstream (verbatim; `Basic.lean`, `Bernstein.lean`).

```
/-- `f` is completely monotone on `(0, ∞)`: `C^∞` there, and `(-1)^n f^{(n)} ≥ 0` there for every `n`. -/
def IsCompletelyMonotoneOn (f : ℝ → ℝ) : Prop :=
  ContDiffOn ℝ (⊤ : ℕ∞) f (Set.Ioi 0) ∧
  ∀ (n : ℕ) (t : ℝ), 0 < t → 0 ≤ (-1 : ℝ) ^ n * iteratedDeriv n f t

/-- **Bridge (exact equivalence).** Complete monotonicity on `(0, ∞)` is upstream's `AdmissiblePotential`. -/
theorem isCompletelyMonotoneOn_iff_admissible {f : ℝ → ℝ} :
    IsCompletelyMonotoneOn f ↔ OAI.AtomicTriangular.AdmissiblePotential f := by
  constructor
  · intro h
    refine ⟨h.1, fun t ht => ?_, h.2⟩
    simpa using h.2 0 t ht
  · intro h
    exact ⟨h.1, h.2.2⟩

/-- `g` is a Bernstein function on `(0, ∞)`: `C^∞`, nonnegative, with completely monotone derivative. -/
def IsBernsteinOn (g : ℝ → ℝ) : Prop :=
  ContDiffOn ℝ (⊤ : ℕ∞) g (Set.Ioi 0) ∧ (∀ t : ℝ, 0 < t → 0 ≤ g t) ∧ IsCompletelyMonotoneOn (deriv g)
```

A function $f : \mathbb{R} \to \mathbb{R}$ is CM if it is C^∞ on $(0, \infty)$ and $(-1)^n f^{(n)}(t) \geq 0$ for all $n \geq 0$ and $t > 0$; g is *Bernstein* if it is C^∞ on $(0, \infty)$, $g \geq 0$ there, and g' is CM. The CM definition is that of Cohn–Kumar [3], and a recent survey-style preprint [4] uses both definitions on $(0, \infty)$ [L, CM-L-0002, CM-L-0004]. Only the values on $(0, \infty)$ matter (`congr`), and the smoothness order is $(\top : \mathbb{N}_\infty)$, i.e. C^∞ and not analytic. The first theorem of Listing 1 says that this definition is *exactly* upstream's admissibility [A, CM-A-0002]: upstream's second clause $g \geq 0$ is the case $n = 0$ of the third.

Mathlib already has a neighbour. The pinned Mathlib (`d13f23b7`) contains `Mathlib/Analysis/Calculus/AbsolutelyMonotone.lean` [7], whose definition `AbsolutelyMonotoneOn f s` (line 57) asks for a Taylor series with nonnegative terms ($f^{(n)} \geq 0$, a sign-free *mirror* of complete monotonicity, not the same notion: f is CM on $(0, \infty)$ exactly when $x \mapsto f(-x)$ is absolutely monotone on $(-\infty, 0)$), and which proves only six theorems, `contDiffOn`, `of_contDiff`, `iff_iteratedDerivWithin_nonneg`, `iteratedDerivWithin_nonneg`, `add` and `smul` [L, CM-L-0001]. It has no product, composition, translation,

Bernstein-function or Laplace-transform lemma. Our definition is therefore *not* a drop-in: a Mathlib contribution would have to be built on it (Section 4).

3 Step-by-step human demonstration

Each step states the fact, proves it, and names the Lean declaration that checks it (Listings 2–5). The prose proofs are [C, CM-C-0001]; the Lean statements are [A]. Throughout, $I = (0, \infty)$ and $h_n(t) := (-1)^n f^{(n)}(t)$.

Listing 2: Closure lemmas (statements verbatim, proofs elided except where shown).

```
/-- Only the values on `(0, ∞)` matter. -/
theorem congr (hf : IsCompletelyMonotoneOn f) (heq : EqOn f g (Ioi 0)) : IsCompletelyMonotoneOn g := -- proof elided

/-- A nonnegative multiple. -/
theorem const_mul (hf : IsCompletelyMonotoneOn f) {c : ℝ} (hc : 0 ≤ c) :
    IsCompletelyMonotoneOn (fun t => c * f t) := -- proof elided

/-- The sum of two completely monotone functions. -/
theorem add (hf : IsCompletelyMonotoneOn f) (hg : IsCompletelyMonotoneOn g) :
    IsCompletelyMonotoneOn (f + g) := -- proof elided

/-- **Product (Leibniz).** The product of two completely monotone functions is completely monotone. -/
theorem mul (hf : IsCompletelyMonotoneOn f) (hg : IsCompletelyMonotoneOn g) :
    IsCompletelyMonotoneOn (f * g) :=
  ⟨hf.1.mul hg.1, fun n t ht => AutoresearchYukawa.mul_sign n (hf.contDiffAt ht n) (hg.contDiffAt ht n)
    (fun i _ => hf.2 i t ht) (fun i _ => hg.2 i t ht)⟩

/-- **Translation.** `t ↦ f (t + c)` is completely monotone for `c ≥ 0`. -/
theorem comp_add_const (hf : IsCompletelyMonotoneOn f) {c : ℝ} (hc : 0 ≤ c) :
    IsCompletelyMonotoneOn (fun t => f (t + c)) := -- proof elided

/-- **Derivative.** `-f'` is completely monotone. -/
theorem neg_deriv (hf : IsCompletelyMonotoneOn f) : IsCompletelyMonotoneOn (fun t => -deriv f t) := -- proof elided

/-- Each `(-1)^n f^{(n)}` is antitone on `(0, ∞)`. -/
theorem antitoneOn_sign (hf : IsCompletelyMonotoneOn f) (n : ℕ) :
    AntitoneOn (fun t => (-1 : ℝ) ^ n * iteratedDeriv n f t) (Ioi 0) := -- proof elided

/-- **Difference of a translate.** `t ↦ f t - f (t + c)` is completely monotone for `c ≥ 0`. -/
theorem sub_comp_add_const (hf : IsCompletelyMonotoneOn f) {c : ℝ} (hc : 0 ≤ c) :
    IsCompletelyMonotoneOn (fun t => f t - f (t + c)) := by
  have hs := hf.comp_add_const hc
  refine ⟨hf.1.sub hs.1, fun n t ht => ?_⟩
  have htc : 0 < t + c := by linarith
  have e : iteratedDeriv n (fun t => f t - f (t + c)) t =
      iteratedDeriv n f t - iteratedDeriv n f (t + c) := by
    have h := iteratedDeriv_sub (hf.contDiffAt ht n) (hs.contDiffAt ht n)
    rw [iteratedDeriv_comp_add_const] at h
    exact h
  rw [e, mul_sub, sub_nonneg]
  exact hf.antitoneOn_sign n ht htc (by linarith)

/-- **Finite nonnegative combinations.** -/
theorem sum {ι : Type*} (s : Finset ι) (w : ι → ℝ) (F : ι → ℝ → ℝ) (hw : ∀ i ∈ s, 0 ≤ w i)
    (hF : ∀ i ∈ s, IsCompletelyMonotoneOn (F i)) :
    IsCompletelyMonotoneOn (fun t => ∑ i ∈ s, w i * F i t) := -- proof elided

/-- `e^{-a t}` is completely monotone for `a ≥ 0`. -/
theorem exp_neg_mul {a : ℝ} (ha : 0 ≤ a) : IsCompletelyMonotoneOn (fun t => Real.exp (-(a * t))) :=
  of_admissible (AutoresearchKeldysh.expNeg_admissible ha)

/-- `t^{-s}` is completely monotone for `s ≥ 0`. -/
theorem rpow_neg {s : ℝ} (hs : 0 ≤ s) : IsCompletelyMonotoneOn (fun t => t ^ (-s)) := -- proof elided
```

Step 1: linear closure [A, CM-A-0003]. Derivatives are linear, so for $c \geq 0$ and $f, g \in \mathrm{CM}$, cf and $f + g$ are CM; finite nonnegative combinations follow by induction (`const_mul`, `add`, `sum`); nonnegative constants are CM (`const`).

Step 2: the Leibniz sign lemma [A, CM-A-0003]. If $(-1)^i u^{(i)} \geq 0$ and $(-1)^i v^{(i)} \geq 0$ for all $i \leq n$, then $(-1)^n (uv)^{(n)} \geq 0$. Indeed $(uv)^{(n)} = \sum_i \binom{n}{i} u^{(i)} v^{(n-i)}$ and $(-1)^n = (-1)^i (-1)^{n-i}$, so each summand is a product of nonnegative numbers. Hence $f, g \in \mathrm{CM} \Rightarrow fg \in \mathrm{CM}$ (`mul`, Listing 2; the lemma `mul_sign` it calls was proved in an earlier module).

Step 3: derivatives, translates and differences [A, CM-A-0003]. Since $h_n' = (-1)^n f^{(n+1)} = -h_{n+1} \le 0$, each h_n is nonincreasing on I (mean value theorem; `antitoneOn_sign`). Consequences: (i) $-f'$ is CM, because $(-1)^n(-f')^{(n)} = h_{n+1} \ge 0$ (`neg_deriv`); (ii) f itself is nonincreasing (`antitoneOn`); (iii) for $c \ge 0$ the translate $t \mapsto f(t+c)$ is CM, because its derivatives are $f^{(n)}(t+c)$ and $t + c > 0$ (`comp_add_const`); (iv) the difference

$$f(t) - f(t+c) \text{ is CM:} \qquad (-1)^n \big[f(t) - f(t+c)\big]^{(n)} = h_n(t) - h_n(t+c) \ge 0$$

because h_n is nonincreasing (`sub_comp_add_const`, shown in full). No integral representation is used.

Step 4: exponentials and powers [A, CM-A-0004]. $\big(e^{-at}\big)^{(n)} = (-a)^n e^{-at}$, so $h_n = a^n e^{-at} \ge 0$ for $a \ge 0$ (`exp_neg_mul`). $\big(t^{-s}\big)^{(n)} = (-s)(-s-1)\cdots(-s-n+1)t^{-s-n}$, so $h_n = s(s+1)\cdots(s+n-1)t^{-s-n} \ge 0$ for $s \ge 0$ (`rpow_neg`). (In Lean these two go through earlier repository modules; the Mathlib-only core of Section 4 re-proves them natively.)

Listing 3: Subordination: the sign statement by strong induction and the general theorem (verbatim, complete).

```
/-- The sign statement behind the composition theorem, by strong induction on the order. -/
theorem comp_sign (hg : IsBernsteinOn g) (hpos : ∀ t, 0 < t → 0 < g t) :
    ∀ n : ℕ, ∀ f : ℝ → ℝ, IsCompletelyMonotoneOn f → ∀ m ≤ n, ∀ t : ℝ, 0 < t →
      0 ≤ (-1 : ℝ) ^ m * iteratedDeriv m (fun t => f (g t)) t := by
  intro n
  induction n with
  | zero =>
    intro f hf m hm t ht
    rw [Nat.le_zero.mp hm]
    simpa using hf.nonneg (hpos t ht)
  | succ n ih =>
    intro f hf m hm t ht
    rcases Nat.lt_or_eq_of_le hm with hlt | rfl
    · exact ih f hf m (Nat.lt_succ_iff.mp hlt) t ht
    · have hF : IsCompletelyMonotoneOn (fun u => -deriv f u) := hf.neg_deriv
      have hEq : EqOn (deriv (fun t => f (g t)))
          (-((fun t => -deriv f (g t)) * deriv g)) (Ioi 0) := by
        intro s hs
        have h1 : HasDerivAt f (deriv f (g s)) (g s) := (hf.differentiableAt (hpos s hs)).hasDerivAt
        have h3 : HasDerivAt (fun t => f (g t)) (deriv f (g s) * deriv g s) s :=
          h1.comp s (hg.hasDerivAt hs)
        rw [h3.deriv]
        simp
      rw [iteratedDeriv_succ', hEq.iteratedDeriv_of_isOpen isOpen_Ioi n ht, iteratedDeriv_neg]
      have hFg : ContDiffOn ℝ (⊤ : ℕ∞) (fun t => -deriv f (g t)) (Ioi 0) :=
        hF.1.comp hg.1 (fun s hs => hpos s hs)
      have key : 0 ≤ (-1 : ℝ) ^ n * iteratedDeriv n ((fun t => -deriv f (g t)) * deriv g) t :=
        AutoresearchYukawa.mul_sign n ((hFg.contDiffAt (Ioi_mem_nhds ht)).of_le (by exact_mod_cast le_top))
          (hg.2.2.contDiffAt ht n) (fun i hi => ih (fun u => -deriv f u) hF i hi t ht)
          (fun i _ => hg.2.2.2 i t ht)
      calc (0 : ℝ) ≤ (-1) ^ n * iteratedDeriv n ((fun t => -deriv f (g t)) * deriv g) t := key
        _ = (-1) ^ (n + 1) * -iteratedDeriv n ((fun t => -deriv f (g t)) * deriv g) t := by ring

/-- **Subordination (general).** A completely monotone function composed with a positive Bernstein function is
completely monotone. -/
theorem IsCompletelyMonotoneOn.comp_bernstein {f g : ℝ → ℝ} (hf : IsCompletelyMonotoneOn f)
    (hg : IsBernsteinOn g) (hpos : ∀ t, 0 < t → 0 < g t) : IsCompletelyMonotoneOn (fun t => f (g t)) :=
  ⟨hf.1.comp hg.1 (fun s hs => hpos s hs), fun n t ht => hg.comp_sign hpos n f hf n le_rfl t ht⟩
```

Step 5: subordination [A, CM-A-0005]. *Claim.* If $f \in$ CM and ψ is Bernstein with $\psi > 0$ on I, then $f \circ \psi \in$ CM. *Proof.* Smoothness is closure of C^∞ under composition. For the sign condition prove, by induction on n, the statement $P(n)$: *for every* CM function f and every $m \le n$, $(-1)^m (f \circ \psi)^{(m)} \ge 0$. $P(0)$ is $f(\psi) \ge 0$. Assume $P(n)$ and let $f \in$ CM. Orders $m \le n$ are covered by $P(n)$. For $m = n+1$ put $F := -f'$, which is CM by Step 3(i). Then $(f \circ \psi)' = f'(\psi)\psi' = -\big(F \circ \psi\big)\psi'$ and

$$(-1)^{n+1}(f \circ \psi)^{(n+1)} = (-1)^n \big[(F \circ \psi)\,\psi'\big]^{(n)} \ge 0$$

by the Leibniz sign lemma (Step 2) with $u = F \circ \psi$ (sign conditions up to order n by $P(n)$ applied to F) and $v = \psi'$ (which is CM by definition of Bernstein). Quantifying over *all* CM f in $P(n)$ is what makes the induction close without the Faà di Bruno formula. □ *The hypothesis* $\psi > 0$. f is constrained only on I, and

$\psi \equiv 0$ is Bernstein, so $f \circ 0$ is the junk value $f(0)$; `junk_comp_zero_not_cm` exhibits a CM function with negative junk value at 0 for which the composition fails [A, CM-A-0013]. The theorem is classical in spirit ("Bochner subordination" [4]); we did not verify where it is stated with a theorem number in the standard monograph [5] [C, CM-C-0005]. Our contribution is only the machine-checked proof on I.

Listing 4: Bernstein instances and consequences (selected statements, verbatim).

```
/-- `t ↦ a t + b` is Bernstein for `a, b ≥ 0`. -/
theorem affine {a b : ℝ} (ha : 0 ≤ a) (hb : 0 ≤ b) : IsBernsteinOn (fun t => a * t + b) := -- proof elided

/-- `t ↦ t^α` is Bernstein for `0 ≤ α ≤ 1`. -/
theorem rpow {α : ℝ} (h0 : 0 ≤ α) (h1 : α ≤ 1) : IsBernsteinOn (fun t => t ^ α) := -- proof elided

/-- `√` is a Bernstein function. -/
theorem sqrt : IsBernsteinOn Real.sqrt := -- proof elided

/-- **Bernstein ∘ Bernstein** (inner function positive) is Bernstein. -/
theorem comp (hf : IsBernsteinOn f) (hg : IsBernsteinOn g) (hpos : ∀ t, 0 < t → 0 < g t) :
    IsBernsteinOn (fun t => f (g t)) := -- proof elided

/-- `t ↦ e^{-a ψ(t)}` is completely monotone for `a ≥ 0` and `ψ` a positive Bernstein function. -/
theorem exp_neg_comp {ψ : ℝ → ℝ} {a : ℝ} (ha : 0 ≤ a) (hψ : IsBernsteinOn ψ) (hpos : ∀ t, 0 < t → 0 < ψ t) :
    IsCompletelyMonotoneOn (fun t => Real.exp (-(a * ψ t))) := -- proof elided

/-- Special case `ψ = √`: `f (√t)` is completely monotone. -/
theorem IsCompletelyMonotoneOn.comp_sqrt {f : ℝ → ℝ} (hf : IsCompletelyMonotoneOn f) :
    IsCompletelyMonotoneOn (fun t => f (Real.sqrt t)) := -- proof elided

/-- **Positive control.** `exp(-κ √t)` is completely monotone (`κ ≥ 0`): a one-line re-derivation of
`AutoresearchYukawa.phi_admissible` (which needed `κ > 0`) via subordination, replacing its bespoke induction. -/
theorem phi_cm {κ : ℝ} (hκ : 0 ≤ κ) : IsCompletelyMonotoneOn (AutoresearchYukawa.phi κ) :=
  ((exp_neg_mul hκ).comp_sqrt).congr' fun t _ => by simp only [AutoresearchYukawa.phi, neg_mul]
```

Step 6: Bernstein instances and closure [A, CM-A-0006]. For $\psi = at + b$ ($a, b \geq 0$), $\psi' = a$ is CM. For $\psi = t^\alpha$, $0 \leq \alpha \leq 1$: $\psi' = \alpha t^{-(1-\alpha)}$ is CM by Step 4; $\psi = \sqrt{t}$ is the case $\alpha = \frac{1}{2}$. Bernstein functions are closed under sums and nonnegative multiples (derivatives add) and under composition with a positive Bernstein function: $(f \circ g)' = (f' \circ g)\, g'$, where $f' \circ g \in \mathrm{CM}$ by Step 5 applied to f' (which is CM by definition of Bernstein) and $g' \in \mathrm{CM}$, then Step 2. Examples: $e^{-a\psi}$ is CM (`exp_neg_comp`); $f(at + b)$ and $f(\sqrt{t})$ are CM; and $e^{-\kappa\sqrt{t}}$ is CM for every $\kappa \geq 0$ (`phi_cm`), a one-line re-derivation of an earlier theorem that needed $\kappa > 0$ and a bespoke induction. Also $1 - e^{-at}$ and $\log(1 + t)$ are Bernstein (Examples module).

Listing 5: Laplace mixtures and locally uniform limits (statements verbatim).

```
/-- All moments from exponential integrability. -/
theorem integrable_pow_mul (hk : Measurable k) (hm : Measurable m) (hk0 : ∀ᵐ x ∂μ, 0 ≤ k x)
    (hm0 : ∀ᵐ x ∂μ, 0 ≤ m x) (hint : ∀ c : ℝ, 0 < c → Integrable (fun x => Real.exp (-(c * k x)) * m x) μ)
    (n : ℕ) {c : ℝ} (hc : 0 < c) : Integrable (fun x => k x ^ n * Real.exp (-(c * k x)) * m x) μ := -- proof elided

/-- **Laplace mixtures (general).** `t ↦ ∫ e^{-t k(x)} m(x) dμ(x)` is completely monotone on `(0, ∞)` when
`k, m ≥ 0` a.e. are measurable and `e^{-c k} m` is integrable for every `c > 0`. -/
theorem laplace (hk : Measurable k) (hm : Measurable m) (hk0 : ∀ᵐ x ∂μ, 0 ≤ k x) (hm0 : ∀ᵐ x ∂μ, 0 ≤ m x)
    (hint : ∀ c : ℝ, 0 < c → Integrable (fun x => Real.exp (-(c * k x)) * m x) μ) :
    IsCompletelyMonotoneOn (fun t => ∫ x, Real.exp (-(t * k x)) * m x ∂μ) := -- proof elided

/-- **Laplace transform of a measure on `[0, ∞)`** (easy direction of Bernstein-Widder). -/
theorem laplace_measure {μ : Measure ℝ} (hμ : ∀ᵐ x ∂μ, 0 ≤ x)
    (hint : ∀ c : ℝ, 0 < c → Integrable (fun x => Real.exp (-(c * x))) μ) :
    IsCompletelyMonotoneOn (fun t => ∫ x, Real.exp (-(t * x)) ∂μ) := -- proof elided

/-- **Locally uniform limits.** -/
theorem of_tendstoLocallyUniformlyOn {F G : ℕ → ℝ → ℝ} {f : ℝ → ℝ}
    (hF : ∀ k, IsCompletelyMonotoneOn (F k))
    (hG : ∀ n : ℕ, TendstoLocallyUniformlyOn (fun k => iteratedDeriv n (F k)) (G n) atTop (Ioi 0))
    (hf : EqOn f (G 0) (Ioi 0)) :
    IsCompletelyMonotoneOn f ∧ ∀ n : ℕ, EqOn (iteratedDeriv n f) (G n) (Ioi 0) := -- proof elided
```

Step 7: Laplace mixtures [A, CM-A-0007]. Let $k, m \geq 0$ be measurable and $f(t) = \int e^{-tk} m\, d\mu$ with $e^{-ck} m$ integrable for every $c > 0$. Since $u^n \leq n!\,(c/2)^{-n} e^{(c/2)u}$ for $u \geq 0$,

$$k^n e^{-ck} m \leq \frac{n!}{(c/2)^n}\, e^{-(c/2)k} m,$$

so every moment $\int k^n e^{-ck} m\, d\mu$ is finite (`integrable_pow_mul`): the earlier lemma assumed all moments, here only $n = 0$ is assumed. This dominates the n-th t-derivative of the integrand uniformly for $t \geq c$, so one may differentiate under the integral and $(-1)^n f^{(n)}(t) = \int k^n e^{-tk} m\, d\mu \geq 0$ (`laplace`). For a measure μ on $[0, \infty)$ whose Laplace transform is finite for every $c > 0$ this says that its Laplace transform is CM (`laplace_measure`), the easy direction of Bernstein–Widder; the converse (quoted in [4] as a standard theorem, going back to Widder [6]) is *not* proved here.

Step 8: limits [A, CM-A-0008]. If $F_k \in \mathrm{CM}$ and, for every n, $F_k^{(n)} \to G_n$ locally uniformly on I, with $f = G_0$ there, then by the classical theorem on uniform convergence of derivatives f is C^∞ with $f^{(n)} = G_n$, and $(-1)^n G_n = \lim (-1)^n F_k^{(n)} \geq 0$ (`of_tendstoLocallyUniformlyOn`); smoothness is derived, not assumed. A pointwise limit without control of derivatives is not covered.

4 The Lean formalization

Table 1: Modules of `formal_cert/Programmes/CompletelyMonotone/` (producer-stage logs, gate `gate.py`): theorems declared, `#print axioms` lines printed, compiler exit code, seconds, first 12 hex digits of the source sha256.

module	decl.	printed	rc	s	sha256
`Basic`	25	25	0	14.2	`eaebc8f68fca`
`Bernstein`	16	16	0	13.5	`9e192c438a94`
`Laplace`	4	4	0	12.8	`fc721ec52eb5`
`Limits`	2	2	0	12.9	`04819006b336`
`Examples`	6	6	0	13.2	`27994bc78842`
`Corollaries`	12	12	0	14.8	`45e2ff5f4e85`
`NegativeControls`	9	9	0	13.2	`83f5de212ee3`
`MathlibOnly`	36	36	0	8.1	`d0da3c865357`
`MathlibOnlyBridge`	4	4	0	12.2	`40c0d8be82ca`
`Applications`	16	16	0	13.5	`a6a0d17048eb`
`Controls` (planted)	–	2	0	12.2	`1469ffd33642`

Gate. The gate (`results/programmes/2026-10-25/P3/gate.py`) reports **OVERALL PASS**: for every library module the exit code is 0, no error, no `sorry` warning and no `sorry`/`admit`/`axiom` token in the source (outside comments); every `theorem`/`lemma` declared has a parsed `#print axioms` entry (130 declared, 130 printed, 114 of them from the first stage and 16 from `Applications`); all of the printed axiom sets equal {`propext`, `Classical.choice`, `Quot.sound`} (declarations off the whitelist: 0); the library logs contain 0 compiler warnings; the `MathlibOnly` module imports only `Mathlib.*`; and each log is newer than its source and was started after its imports' logs finished. Each module compiled in 8.1–14.8 s in the final dependency-ordered rebuild [A, CM-A-0002 to CM-A-0019]. Re-running `gate.py` on the stored logs for this paper left `gate.json` with identical content. (The gate's own bugs found along the way—primed names not parsed, backtick-quoted `sorry` warning, a docstring line mistaken for a declaration—were fixed; before the final rebuild the gate correctly reported stale logs.) Selected axiom lines, verbatim:

```
'ProgCM.isCompletelyMonotoneOn_iff_admissible' depends on axioms: [propext, Classical.choice, Quot.sound]
'ProgCM.IsCompletelyMonotoneOn.mul' depends on axioms: [propext, Classical.choice, Quot.sound]
'ProgCM.IsCompletelyMonotoneOn.sub_comp_add_const' depends on axioms: [propext, Classical.choice, Quot.sound]
'ProgCM.triangular_optimal' depends on axioms: [propext, Classical.choice, Quot.sound]
'ProgCM.IsBernsteinOn.comp_sign' depends on axioms: [propext, Classical.choice, Quot.sound]
'ProgCM.IsCompletelyMonotoneOn.comp_bernstein' depends on axioms: [propext, Classical.choice, Quot.sound]
'ProgCM.laplace' depends on axioms: [propext, Classical.choice, Quot.sound]
'ProgCM.laplace_measure' depends on axioms: [propext, Classical.choice, Quot.sound]
'ProgCM.of_tendstoLocallyUniformlyOn' depends on axioms: [propext, Classical.choice, Quot.sound]
'ProgCM.dipolar_cm' depends on axioms: [propext, Classical.choice, Quot.sound]
'ProgCM.yukawa_cm' depends on axioms: [propext, Classical.choice, Quot.sound]
'ProgCM.keldysh_cm' depends on axioms: [propext, Classical.choice, Quot.sound]
'ProgCM.Applications.quadrupolarPair_cm' depends on axioms: [propext, Classical.choice, Quot.sound]
'ProgCM.Applications.quadrupoleC_not_cm' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Planted controls [A, CM-A-0020]. `Controls.lean`, imported by nothing, "proves" the false statement that cos is CM twice, once with `sorry` and once with a planted axiom; `cos_not_cm` refutes it. Both compile with exit code 0 and only `#print axioms` exposes them:

Listing 6: Planted controls: source (top) and compiler output (bottom, producer stage; rc and seconds first).

```
theorem ProgCM.ctl_sorry : ProgCM.IsCompletelyMonotoneOn Real.cos := sorry
axiom ProgCM.ctl_cheat : False
theorem ProgCM.ctl_axiom : ProgCM.IsCompletelyMonotoneOn Real.cos := ProgCM.ctl_cheat.elim
#print axioms ProgCM.ctl_sorry
#print axioms ProgCM.ctl_axiom
```

```
rc = 0, 12.2 s
formal_cert/Programmes/CompletelyMonotone/Controls.lean:11:8: warning: declaration uses `sorry`
'ProgCM.ctl_sorry' depends on axioms: [propext, sorryAx, Classical.choice, Quot.sound]
'ProgCM.ctl_axiom' depends on axioms: [propext, Classical.choice, ProgCM.ctl_cheat, Quot.sound]
```

Paper-stage recompile [X, CM-X-0003]. For this paper, `Bernstein.lean` (rc 0, 15.2 s, 16 axiom lines), `Applications.lean` (rc 0, 12.6 s, 16 lines) and `Controls.lean` (rc 0, 13.2 s) were elaborated again under the pinned stack without writing oleans. Their printed output is identical to (resp. identical to, identical to) the producer-stage output. This is the *producing pipeline*, imports the producer's oleans for `Basic`, `Laplace`, `Corollaries` and the rest, and covers 3 of the 11 files: it is not producer $\neq$ verifier. The independent rebuild of all 11 files by a separate agent is the audit described in the disclosure above and in Section 8; it is reported here, not filed as a ledger claim.

Mathlib-only core [A, CM-A-0019]. `Basic` imports repository modules and hence upstream openai/math. `MathlibOnly.lean` (namespace `ProgCMCore`, 36 theorems, 8 `Mathlib.*` imports) re-proves the Leibniz lemma, e^{-at}, t^{-s}, the Laplace analytic core (with a `LaplaceData` structure that carries only the weakened integrability hypothesis), the closure lemmas, the Bernstein instances and `comp_bernstein`. `MathlibOnlyBridge.lean` (4 theorems) proves that its two definitions are those of the main library by `Iff.rfl`. It duplicates code and omits limits, examples, Bernstein closure, `laplace_measure` and the corollaries.

Mathlib readiness (not a claim) [C, CM-C-0006]. A pull request would have to rebase on `AbsolutelyMonotoneOn` (Section 2), be set-parametric, rename `IsBernsteinOn` (Mathlib already uses `bernstein` for Bernstein polynomials), remove the duplication, adopt the module system, shorten overlong lines (22 in `MathlibOnly.lean`, 24 in `Basic.lean`) and pass the Mathlib linters, *which were not run.* Mathlib's contribution policy says, in the fragments we could read, that LLM-written comments on GitHub or Zulip are not allowed, that LLM use must be explained in the PR description, and that AI-written code without the supervision of a Lean subject expert falls short of the bar "by a large margin" [8] [L, CM-L-0005] (the fetch tool returned only short quotations, so the full section was not read word for word). The route to human review of this library therefore runs through a Lean expert, not through an agent. No PR was opened.

5 Applications

Listing 7: H1, H2, H3 and new potentials as corollaries (`Corollaries.lean`, `Applications.lean`; statements verbatim).

```
/-- Riesz `t^{-s/2}` (energy `Σ r^{-s}`), `s ≥ 0`. -/
theorem riesz_cm {s : ℝ} (hs : 0 ≤ s) : IsCompletelyMonotoneOn (TriangularRiesz.riesz s) :=
  (rpow_neg (by linarith : 0 ≤ s / 2)).congr' fun _ _ => rfl

/-- **H2 as a corollary.** Planar Yukawa `e^{-κ r}/r` (`κ ≥ 0`): Riesz × (exp ∘ √). -/
theorem yukawa_cm {κ : ℝ} (hκ : 0 ≤ κ) : IsCompletelyMonotoneOn (AutoresearchYukawa.g κ) :=
  ((riesz_cm zero_le_one).mul (phi_cm hκ)).congr (AutoresearchYukawa.g_eqOn κ)

/-- **H1 as a corollary.** Bilayer dipolar exciton `2 (1/r - 1/√(r^2 + d^2))`, every `d`: difference of a translate. -/
theorem dipolar_cm (d : ℝ) : IsCompletelyMonotoneOn (AutoresearchDipolar.g d) :=
  (((rpow_neg (by norm_num : (0 : ℝ) ≤ 1 / 2)).sub_comp_add_const (sq_nonneg d)).const_mul
    zero_le_two).congr' fun _ _ => rfl

/-- Screened multipole differences `r^{-s} - (r^2 + c)^{-s/2}`, `s ≥ 0`, `c ≥ 0`. -/
theorem shiftDiff_cm {s c : ℝ} (hs : 0 ≤ s) (hc : 0 ≤ c) :
    IsCompletelyMonotoneOn (fun t => TriangularRiesz.riesz s t - TriangularRiesz.riesz s (t + c)) :=
  (riesz_cm hs).sub_comp_add_const hc

/-- **H3L as a corollary of the weakened Laplace lemma** (only the `n = 0` integrability is used). -/
```

```
theorem keldysh_cm {r0 : ℝ} (hr0 : 0 < r0) : IsCompletelyMonotoneOn (AutoresearchKeldysh.keldysh r0) := -- proof elided

/-- Finite-size (soft-core) Yukawa `e^{-κ √(r^2 + b^2)} / √(r^2 + b^2)`: translation closure. -/
theorem yukawa_softcore_cm {κ b : ℝ} (hκ : 0 ≤ κ) :
    IsCompletelyMonotoneOn (fun t => AutoresearchYukawa.g κ (t + b ^ 2)) :=
  (yukawa_cm hκ).comp_add_const (sq_nonneg b)

/-- Stretched exponential `e^{-a r^{2α}}` (`a ≥ 0`, `0 ≤ α ≤ 1`, i.e. `e^{-a t^α}`): subordination. -/
theorem stretchedExp_cm {a α : ℝ} (ha : 0 ≤ a) (h0 : 0 ≤ α) (h1 : α ≤ 1) :
    IsCompletelyMonotoneOn (fun t => Real.exp (-(a * t ^ α))) :=
  exp_neg_comp ha (IsBernsteinOn.rpow h0 h1) (fun _ ht => Real.rpow_pos_of_pos ht α)

/-- Triangular optimality for the soft-core Yukawa interaction (conditional on upstream). -/
theorem triangular_yukawa_softcore_optimal {κ b : ℝ} (hκ : 0 ≤ κ) (C : Set OAI.AtomicTriangular.Plane)
    (hC : OAI.AtomicTriangular.LocallyFinite C) (hd : OAI.AtomicTriangular.DensityOne C) :
    OAI.AtomicTriangular.latticeEnergy (fun t => AutoresearchYukawa.g κ (t + b ^ 2)) ≤
      OAI.AtomicTriangular.energy (fun t => AutoresearchYukawa.g κ (t + b ^ 2)) C := -- proof elided
```

Listing 8: The three original theorems re-derived with their original statements, and two new results (`Applications.lean`).

```
/-- **H2** (original statement of `AutoresearchYukawa.yukawa_admissible`). -/
theorem H2_yukawa_admissible {κ : ℝ} (hκ : 0 < κ) : AdmissiblePotential (AutoresearchYukawa.g κ) :=
  (((riesz_cm zero_le_one).mul (phi_cm hκ.le)).congr (AutoresearchYukawa.g_eqOn κ)).admissible

/-- **H1** (statement of `AutoresearchDipolar.dipolar_admissible_all`; the original `dipolar_admissible` needed `0 < d`). -/
theorem H1_dipolar_admissible (d : ℝ) : AdmissiblePotential (AutoresearchDipolar.g d) :=
  ((((rpow_neg (by norm_num : (0 : ℝ) ≤ 1 / 2)).sub_comp_add_const (sq_nonneg d)).const_mul
    zero_le_two).congr' fun _ _ => rfl).admissible

/-- **H3L** (original statement of `AutoresearchKeldysh.keldysh_admissible`). -/
theorem H3_keldysh_admissible {r0 : ℝ} (hr0 : 0 < r0) : AdmissiblePotential (AutoresearchKeldysh.keldysh r0) :=
  (keldysh_cm hr0).admissible

/-- The screened bilayer exciton potential is completely monotone (`κ ≥ 0`, every `d`). -/
theorem screenedDipolar_cm {κ : ℝ} (hκ : 0 ≤ κ) (d : ℝ) : IsCompletelyMonotoneOn (screenedDipolar κ d) :=
  ((yukawa_cm hκ).sub_comp_add_const (sq_nonneg d)).const_mul zero_le_two

/-- **Yukawa mixtures with a nonnegative spectral density are completely monotone**, provided
`e^{-cκ} m(κ)` is integrable on `(0, ∞)` for every `c > 0`. Proof: `t^{-1/2} · (Laplace(m) ∘ √)`. -/
theorem yukawaSpectrum_cm {m : ℝ → ℝ} (hm : Measurable m) (hm0 : ∀ κ, 0 < κ → 0 ≤ m κ)
    (hint : ∀ c : ℝ, 0 < c → IntegrableOn (fun κ => Real.exp (-(c * κ)) * m κ) (Ioi 0)) :
    IsCompletelyMonotoneOn (yukawaSpectrum m) := -- proof elided

/-- Second difference in `t = r^2` (shifts `0, c, 2c`): completely monotone for completely monotone `f`.
This is NOT the equally spaced-layer quadrupole (shifts `0, d^2, 4d^2`); see Part 3 and Part 4 (`quadrupolarPair_cm`). -/
theorem secondDiff_cm {f : ℝ → ℝ} (hf : IsCompletelyMonotoneOn f) {c : ℝ} (hc : 0 ≤ c) :
    IsCompletelyMonotoneOn (fun t => f t - 2 * f (t + c) + f (t + 2 * c)) :=
  ((hf.sub_comp_add_const hc).sub_comp_add_const hc).congr' fun t _ => by
    show f t - f (t + c) - (f (t + c) - f (t + c + c)) = _
    rw [show t + c + c = t + 2 * c by ring]
    ring
```

H1, H2, H3 [A, CM-A-0010, CM-A-0014]. Writing $t = r^2$: *Dipolar*: $g_d(t) = 2(t^{-1/2} - (t+d^2)^{-1/2}) = 2[f(t) - f(t+d^2)]$ with $f = t^{-1/2}$, CM by Steps 3–4, for every real d (the first version of the theorem needed $d > 0$; a later module already covered every d). *Yukawa*: $g_\kappa(t) = t^{-1/2} \cdot e^{-\kappa\sqrt{t}}$, a product (Step 2) of CM functions (Steps 4, 6), now for $\kappa \geq 0$. *Rytova–Keldysh*: a mixture of Gaussians in the companion paper [14] and hence a Laplace mixture (Step 7); the hard erfcx and integrability bounds are reused from there, and the Lean proof still takes its general-n integrability bound from that module, so the weakened hypothesis is *available* but not *exercised* to shorten anything. Each theorem is 2–3 lines through the library.

New consequences [A, CM-A-0011, CM-A-0015]. Soft-core Yukawa $g_\kappa(t+b^2)$ (translation), stretched exponential e^{-at^α}, $0 \leq \alpha \leq 1$ (subordination by t^α), the Yukawa-screened bilayer exciton $2(g_\kappa(t) - g_\kappa(t+d^2))$, and continuous Yukawa mixtures $\int_{\kappa>0} m(\kappa) g_\kappa(t)\, d\kappa$ for measurable $m \geq 0$ with $e^{-c\kappa}m$ integrable for all $c > 0$ (a Laplace mixture times $t^{-1/2}$ composed with $\sqrt{t}$). For $m = e^{-\kappa}$ the potential equals $1/(r(1+r))$ by an elementary integral [C, CM-C-0004], not checked in Lean. A screen of the repository found no earlier statement of these; no literature search was made, so no novelty is claimed.

Listing 9: Quadrupoles: the field of one quadrupole is not CM, the interaction of two is (`Applications.lean`).

```
/-- The charges `+1, -2, +1` in layers `0, d, 2d` give shifts `0, d^2, 4d^2` in `t = r^2`; unit case `d = 1`. -/
```

```
noncomputable def quadrupole (t : ℝ) : ℝ := t ^ (-(1 / 2 : ℝ)) - 2 * (t + 1) ^ (-(1 / 2 : ℝ)) + (t + 4) ^ (-(1 / 2 : ℝ))

/-- **Negative result.** The (unit) charge-quadrupole combination is not completely monotone. -/
theorem quadrupole_not_cm : ¬ IsCompletelyMonotoneOn quadrupole := fun h => by
  have := h.nonneg (t := 3) (by norm_num)
  linarith [quadrupole_neg_at_three]

/-- **Negative result at every layer spacing** `c > 0`: reduced to `c = 1` by the scaling `t ↦ c t`
(`comp_affine`). -/
theorem quadrupoleC_not_cm {c : ℝ} (hc : 0 < c) : ¬ IsCompletelyMonotoneOn (quadrupoleC c) := -- proof elided

/-- Pair interaction of two aligned `+1, -2, +1` quadrupoles for a pair potential `f` (in `t = r^2`, `c = d^2`). -/
noncomputable def quadrupolarPair (f : ℝ → ℝ) (c t : ℝ) : ℝ := 2 * (3 * f t - 4 * f (t + c) + f (t + 4 * c))

/-- **Quadrupolar exciton-exciton interaction** is completely monotone for every completely monotone pair potential. -/
theorem quadrupolarPair_cm {f : ℝ → ℝ} (hf : IsCompletelyMonotoneOn f) {c : ℝ} (hc : 0 ≤ c) :
    IsCompletelyMonotoneOn (quadrupolarPair f c) := by
  have hg := hf.sub_comp_add_const hc
  have h1 := hg.sub_comp_add_const hc
  have h2 := hg.sub_comp_add_const (by linarith : (0 : ℝ) ≤ 2 * c)
  have h3 := hg.sub_comp_add_const (by linarith : (0 : ℝ) ≤ 3 * c)
  refine (((h1.add h2).add h3).const_mul zero_le_two).congr' fun t _ => ?_
  simp only [quadrupolarPair, Pi.add_apply]
  ring_nf

/-- Unscreened (Coulomb pair potential `t^{-1/2}`) quadrupolar exciton interaction, `c = d^2`. -/
theorem quadrupolarCoulomb_cm (d : ℝ) :
    IsCompletelyMonotoneOn (quadrupolarPair (fun t => t ^ (-(1 / 2 : ℝ))) (d ^ 2)) :=
  quadrupolarPair_cm (rpow_neg (by norm_num : (0 : ℝ) ≤ 1 / 2)) (sq_nonneg d)
```

Quadrupoles: a negative and a positive result [A, CM-A-0016, CM-A-0017; C, CM-C-0002]. Take charges $+1, -2, +1$ in layers $0, d, 2d$, pair interaction $f(r^2 + \Delta z^2)$ and $c = d^2$. *(a)* The field of *one* quadrupole on a single test charge is $f(t) - 2f(t + c) + f(t + 4c)$ (weights $1, -2, 1$, shifts $0, c, 4c$). For $f = t^{-1/2}$ it is *not* CM for any $c > 0$: at $t = 3$, $c = 1$ it equals $1/\sqrt{3} - 1 + 1/\sqrt{7} < 0$ (using $\sqrt{3} > 1.7$, $\sqrt{7} > 2.6$), and scaling $t \mapsto ct$ extends this to every c (`quadrupole_not_cm`, `quadrupoleC_not_cm`). The weights have zero mean but a nonzero second moment, so the far field is $-d^2/r^3 < 0$ [C]. *(b)* The *interaction of two aligned quadrupoles* counts pairs: $\Delta z = 0$ gives $1 + 4 + 1 = 6$, $|\Delta z| = d$ gives $4(-2) = -8$, $|\Delta z| = 2d$ gives $2 \cdot 1 = 2$, hence $6f(t) - 8f(t + c) + 2f(t + 4c) = 2\big(3f(t) - 4f(t + c) + f(t + 4c)\big)$ [C, CM-C-0002]. This *is* CM for every CM f and $c \geq 0$: with $g = f - f(\cdot + c)$ it equals $2[\Delta_c g + \Delta_{2c} g + \Delta_{3c} g]$, $\Delta_a h = h - h(\cdot + a)$, a sum of differences of translates (Step 3(iv); `quadrupolarPair_cm`, Listing 9). In Laplace language $f = \int e^{-ut}\, d\mu(u)$ turns the combination into e^{-ut} times the polynomial $p(x) = 2(3 - 4x + x^4) = 2(1 - x)^2(x^2 + 2x + 3) \geq 0$ in $x = e^{-uc} \in (0, 1]$, whereas the field of one quadrupole gives $1 - 2x + x^4$, negative near $x = 0.79$ (Figure 1).

Triangular optimality, conditional [A, CM-A-0012, CM-A-0018; C, CM-C-0003]. `triangular_optimal` restates upstream: for $g \in \mathrm{CM}$ and every locally finite density-one C, `latticeEnergy g <= energy g C`; it is instantiated for the soft-core Yukawa, stretched exponential, screened dipolar, Yukawa-spectrum and Coulomb quadrupolar potentials. All of these rest on upstream [A, CM-A-0001], which is AI-generated and not peer reviewed, and on the fidelity of its definitions. We prove no finiteness of the optimal energy for these potentials, so for slowly decaying tails the statement can read $\top \leq \top$. Physical modelling (rigid, aligned, classical point charges, no screening beyond the Yukawa factor, no quantum effects) is Tier C and the statements say nothing about uniqueness.

Negative controls [A, CM-A-0013]. Listing 10 shows that each of the listed hypotheses bites (the integrability and measurability hypotheses of `laplace` and $c \geq 0$ in the translation lemma are *not* covered by a control): cos is not CM; $\sqrt{t}$ is Bernstein but not CM and e^{-t} is CM but not Bernstein, so the classes differ; $1/t$ is CM and positive but not Bernstein and $e^{-1/t}$ is not CM, so the inner function must be Bernstein; the hypothesis $\psi > 0$ cannot be dropped; and $e^{-t} - 2e^{-2t}$ is not CM, so mixture weights must be nonnegative.

Listing 10: Negative controls (selected statements, verbatim, `NegativeControls.lean`).

```
theorem cos_not_cm : ¬ IsCompletelyMonotoneOn Real.cos := -- proof elided

theorem sqrt_not_cm : ¬ IsCompletelyMonotoneOn Real.sqrt := -- proof elided

theorem expNeg_not_bernstein : ¬ IsBernsteinOn (fun t => Real.exp (-(1 * t))) := -- proof elided
```

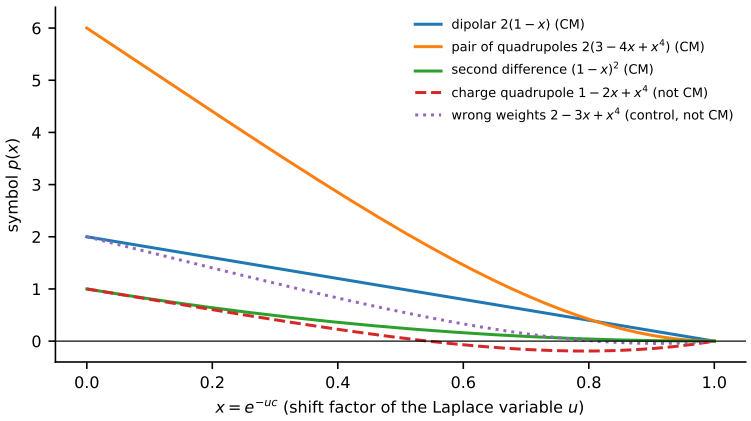


Figure 1: Tier X illustration (CM-X-0004), code-generated; the minima are floating-point grid values. Laplace symbols $p(x)$, $x = e^{-uc}$, of the combinations of translates. Reading $p \geq 0$ on $(0, 1]$ as "CM" uses the easy direction of Bernstein–Widder (`laplace_measure`) together with the representation $t^{-1/2} = \int e^{-ut} u^{-1/2} \pi^{-1/2} du$; reading a negative p as "not CM" would need the hard direction and uniqueness of the representing measure, which are *not* proved here. The Lean proofs do not use symbols: positive cases use differences of translates (Step 3) and `quadrupole_not_cm` is a direct evaluation at $t = 3$. The minima on the plotted grid are -0.1905 at $x = 0.79$ (charge quadrupole) and -0.0443 at $x = 0.91$ (control with wrong weights $2, -3, 1$).

```
theorem expNegInv_not_cm : ¬ IsCompletelyMonotoneOn (fun t : ℝ => Real.exp (-(1 * t⁻¹))) := -- proof elided

theorem junk_comp_zero_not_cm : ¬ IsCompletelyMonotoneOn (fun t => junk ((fun _ : ℝ => (0 : ℝ)) t)) := -- proof elided

theorem signedMix_not_cm : ¬ IsCompletelyMonotoneOn AutoresearchKeldysh.signedMix := -- proof elided
```

6 Numerics and tooling (Tier X)

Everything here is [X]: it illustrates, screens or measures, and supports no claim.

A 60-digit screen with controls [X, CM-X-0001]. For 14 potentials the script `cm_numeric_check.py` evaluates $(-1)^n F^{(n)}(t)$ for $n = 0, \ldots, 7$ at 15 values of t from 10^{-3} to 10^4 with mpmath (60 digits, numerical differentiation) and reports the minimum signed value. A negative value refutes CM up to differentiation error; a clean grid never proves it. *Controls.* The two positive controls ($t^{-1/2}$, $e^{-\sqrt{t}}/\sqrt{t}$) show no negative value; the four negative controls ($\cos t$, $t^{-1/2} - 2(t+1)^{-1/2}$, $g_1 - 2g_2$ and the wrong-weights $2f(t) - 3f(t+1) + f(t+4)$) all show one, so the screen can say no, including for a defect as small as -0.0048. *Results.* Of the 8 candidates, 1 shows a negative value: the one-quadrupole field, with minimum -0.0443 at $n = 0$, $t = 3.16$ on this (mpmath) grid; the Lean value at $t = 3$ is $1/\sqrt{3} - 1 + 1/\sqrt{7} \approx -0.0447$, and on the finer numpy grid of Figure 2 the minimum is -0.0449 at $t \approx 2.79$ (a different grid, not a different quantity). The seven other candidates, including the pair interaction, show none, which is consistent with, and never a proof of, their CM status (the proofs are the Lean theorems).

Proof-length measurement and its reading [X, CM-X-0002]. Method: Lean computes, from the kernel terms, the transitive closure of declarations in `Programmes.*` and `Autoresearch.*` used by each root theorem. Mathlib, upstream `OAI.*` and the repository's `Triangular.*` modules are excluded, on both sides. The script `proof_length.py` counts non-blank, non-comment source lines in each declaration's range; overlapping structure and field ranges are counted twice, on both sides. Neither choice changes the direction of the comparison. The reading of the counts given below (why the closure grows) is an explanation of a measurement, so it is filed with the measurement as Tier X and is not a Tier C claim. Headline proof (lines, original → library): H2 2→2, H1 for $d > 0$ 23→3 (the 23-line proof was the first version), H1 for every d 3→3,

Table 2: The screen. The minimum signed value of $(-1)^n F^{(n)}(t)$ over $n \le 7$ and the 15 grid values of t; for clean rows it is just the smallest positive value (at the grid end), so only the last column is informative.

F	role	min signed	n	t	negative?
$t^{-1/2}$	control (CM)	1.06×10^{-27}	7	10^4	no
$e^{-\sqrt{t}}/\sqrt{t}$	control (CM)	3.84×10^{-62}	7	10^4	no
$\cos t$	control (not CM)	-1.0000	2	10^{-3}	yes
$t^{-1/2} - 2(t+1)^{-1/2}$	control (not CM)	-0.4180	0	3.16	yes
$g_1 - 2g_2$	control (not CM)	-3.34×10^{25}	7	10^{-3}	yes
$t^{-1/2} - 2(t+1)^{-1/2} + (t+4)^{-1/2}$	candidate	-0.0443	0	3.16	yes
$2(3f(t) - 4f(t+1) + f(t+4))$, $f = t^{-1/2}$	candidate	8.06×10^{-33}	7	10^4	no
same with $f = g_1$	candidate	1.35×10^{-65}	7	10^4	no
$2f(t) - 3f(t+1) + f(t+4)$, $f = t^{-1/2}$	control (not CM)	-0.0048	0	10	yes
$f(t) - 2f(t+1) + f(t+2)$, $f = t^{-1/2}$	candidate	6.72×10^{-34}	7	10^4	no
$2(g_1(t) - g_1(t+1))$	candidate	4.15×10^{-64}	7	10^4	no
$2(g_3(t) - g_3(t+4))$	candidate	1.15×10^{-146}	7	10^4	no
$1/(\sqrt{t}(1+\sqrt{t}))$	candidate	4.89×10^{-29}	7	10^4	no
$(1 - e^{-2\sqrt{t}})/t$	candidate	5.04×10^{-29}	7	10^4	no

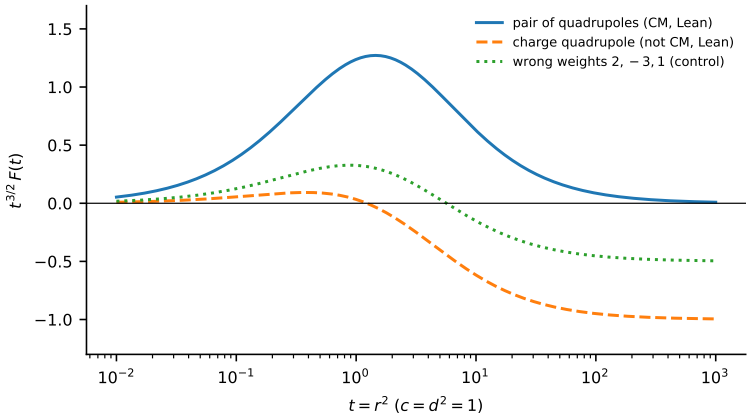


Figure 2: Tier X, code-generated (numpy, 600 log-spaced t). $t^{3/2}F(t)$ for the interaction of two quadrupoles (Lean: CM), the field of one quadrupole (Lean: not CM) and the wrong-weights control. At $t = 10^3$ the values are 0.009 (pair), -0.995 (one quadrupole; the far-field limit is -1, from $t^{-1/2} - 2(t+1)^{-1/2} + (t+4)^{-1/2} \approx 2f'(t) = -t^{-3/2}$) and about -0.5 (control); these are values at the grid end, not limits.

H3 8→2. End to end, however, the closures grow (Figure 3): H2 84 (13 declarations) → 274 (50), H1 72 (12) → 95 (17), H3 250 (34) → 304 (43); the union of the three goes from 406 lines in 59 declarations to 505 in 77, and with the screened-dipolar and Yukawa-spectrum results to 527 (82) and with the two quadrupolar interactions to 548 (87). The cause is that `Basic.exp_neg_mul` routes through the earlier Laplace core, which then enters the closure of every exponential-based result; the Mathlib-only core would avoid it but was not used. The library pays off at the margin: the two new groups of results add 22 and 21 lines. **We therefore do not claim shorter proofs overall.**

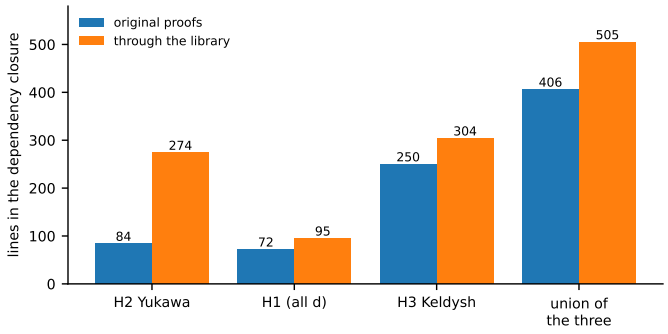


Figure 3: Tier X, code-generated from `proof_length.json`: lines in the dependency closure, original proofs versus through the library.

rusty-SUNDIALS and LeanFlow: no change in this lane. P3 is a Lean infrastructure lane. Complete monotonicity is a symbolic property, so a CVODE integration would test nothing here, and the only numerics are the screen above, written directly in mpmath. No branch of rusty-SUNDIALS exists for P3, no change to LeanFlow was made, and there are therefore no commit SHAs to report; the AutoevolveAI files are uncommitted on base commit `9d24574` of this worktree (the sha256 values of the Lean sources are in Section 9). Improving rusty-SUNDIALS or LeanFlow *via* this library (for example an interval-arithmetic CM check) is future work, not a result.

7 Claim ledger

Every claim above is filed in `ledger/ledger.json` (35 claims: 20 Tier A, 5 L, 6 C, 4 X) with a content-addressed evidence blob (verbatim axiom lines and source hashes for A; verbatim excerpts for L; the argument for C; result files with `claim_licence=false` for X). The Elenchus gate `tools/ledger.py` was run with `-evidence-dir` (`ledger/gate_output.txt`): **0 blocking findings and 20 flags, exit code 1**; the non-zero exit code is due to the flags `LEDGER_UNAUDITED_TIER_A` alone, one for each Tier A row whose statement adequacy no person has audited. A first run found one blocking tier inversion: a Tier C row (the reading of the proof-length counts) depended on a Tier X measurement. The first repair deleted the dependency edge, which only hid the inversion; the audit restored the edge and the gate then blocked again. The reading is now merged into the Tier X row CM-X-0002, so no Tier C row depends on a Tier X row. The gate checks bookkeeping and cannot say that a claim is true. Of the five Tier L rows, only the Salazar Theorem 2.3 excerpt is a theorem statement; `citation_check.py` accepted it and refused a tampered control. CM-L-0001, 0002, 0003 and 0005 are definitions, prose or policy text, are Tier L by convention, and were not run through `citation_check.py`. `elenchus_check.py` could not run on the Lean files (default toolchain, no project search path: COMPILE_ERROR) and skipped the `.py` files (no `CONTROL_INPUTS`); it gave no verdict, which is not a pass. Of the claims, the upstream theorem (CM-A-0001) and every optimality statement (CM-A-0012, CM-A-0018) are conditional on AI-generated, unreviewed work.

8 Limitations

- **No human audit.** The proofs and statements were produced by one AI pipeline and then rebuilt and axiom-checked by an independent AI audit (all 11 files, 167 constants, 0 off the allowlist, planted controls caught; see the disclosure). That audit did not rebuild the upstream openai/math or Mathlib oleans. No person has audited statement adequacy: the 20 Tier A rows keep their `LEDGER_UNAUDITED_TIER_A` flags.
- **Conditionality.** All optimality corollaries depend on upstream openai/math (AI-generated, not peer reviewed); Keldysh-type tails make them read $\top \leq \top$.
- **Preregistration versus outcome** (P3, committed before the attempt): core closure (prior 0.8) done; composition with Bernstein functions (prior 0.5) done; Fourier criterion (prior 0.3) not attempted; Stieltjes mixtures not done; the "Mathlib-PR-ready branch" goal was not met; the kill rule "two budgets per lemma" is not reported on (no lemma is recorded as having been abandoned).
- **Scope of the library.** Domain $(0, \infty)$ only; C^∞, not analytic; no hard Bernstein–Widder, no pointwise limits, no arbitrary CM mixtures, no Stieltjes mixtures, no Fourier criterion (preregistered, prior 0.3, not attempted).
- **Not a Mathlib contribution.** A different formalisation (`AbsolutelyMonotoneOn`) already exists; the main library imports repository and upstream modules; the Mathlib-only core duplicates code and covers only part; linters were not run; names were checked only against the pinned Mathlib.
- **No shorter proofs end to end** (Section 6); the weakened Laplace hypothesis is not exercised by H3.
- **No novelty claim.** Closure properties are elementary; no literature search for novelty was made; subordination is attributed, not located in the monograph [5].
- **Physics** (quadrupoles, screened excitons) is rigid, aligned, classical and unscreened apart from the Yukawa factor; the pair counting is Tier C.
- **Tier X** evidence is a screen with 60-digit numerical differentiation on a coarse grid; it never proves CM.
- **No rusty-SUNDIALS or LeanFlow improvement** was made in this lane.
- **Infrastructure.** An anecdotal first cold compile of `Basic` took 872.6 s (disk contention; the figure comes from the lane's `LIBRARY.md` only, its log was overwritten, and it supports no claim); the Lean files are uncommitted in the repository.

9 Reproducibility

Toolchain: Lean 4.34.1, Mathlib `d13f23b723b8a846827a245b89c10fc7d3f11612`, openai/math `fd4aeeb2ee4fc729c18d98444fed42fd0529eeeb` (pinned copy). AutoevolveAI base commit `9d245742c1655852103abb28f585e21af54851e6`; the files below are uncommitted at the time of writing.

- `Basic.lean`: `eaebc8f68fcab5cc53719f8e7c90ef9e2e4ffe0ce2235a792808c2c85ee10c52`
- `Bernstein.lean`: `9e192c438a949b6d76403135b295a82f93d9918ed00121c630487e9dd909f39d`
- `Laplace.lean`: `fc721ec52eb51e97523e6c07b4d1b3624f608254c28b0a9477f09809c0aa952c`
- `Limits.lean`: `04819006b336f37ca54f6e8459d29c7d9461e17462985ea2f257110282ba78fc`
- `Examples.lean`: `27994bc788426556f45c62674412e36242e108d0e7ea6302506dbc8f8a6a9d39`
- `Corollaries.lean`: `45e2ff5f4e8562f71ad4d36d1c85e384fe3c23060ef01b63e876b93972ed319f`
- `NegativeControls.lean`: `83f5de212ee3f6ad2f2cfb169238c0e1b80a0d3c79af91f69187a47a1661772e`
- `MathlibOnly.lean`: `d0da3c865357431db3817ae7f800c89e9f1289103cdbb7f08fb2184e4b32685d`
- `MathlibOnlyBridge.lean`: `40c0d8be82caacc70e83ba25bfce46fb25e2bc0a31e3f26c27cbdf172cc6e7e0`
- `Applications.lean`: `a6a0d17048eb3e2dd56ee0837e5f688f027b7beecb6cd275e5b5ceb45dceb83f`
- `Controls.lean`: `1469ffd336424ef3dc784a5d65281e4c3e1d943563083cb622c083690c435fce`

```
taskset -c 4-7 python3 scripts/certified_numerics/lean_pinned.py compile formal_cert/Programmes/CompletelyMonotone/<M>.lean \
  --olean Programmes.CompletelyMonotone.<M> --json-out results/programmes/2026-10-25/P3/compile_logs/<M>.json
  (order: Basic; Bernstein, Laplace, Limits; Examples, Corollaries; NegativeControls, MathlibOnly; MathlibOnlyBridge; Applications;
   Controls without --olean)
python -I results/programmes/2026-10-25/P3/gate.py
python -I results/programmes/2026-10-25/P3/applications/cm_numeric_check.py out.json     # Tier X screen
python -I papers/prog_cm_bernstein_library/gen_inputs.py                                  # numbers, listings, figures
python -I papers/prog_cm_bernstein_library/ledger/build_ledger.py ; python -I papers/prog_cm_bernstein_library/ledger/run_gate.py
pdflatex main.tex (twice)
```

Paper-stage recompile logs are in `ledger/recompile/`. All code, data and logs are in this directory and in `results/programmes/2026-10-25/P3/`.

References

[1] OpenAI, *Universal optimality of the triangular lattice*, OpenAI Math release, github.com/openai/math, commit `fd4aeeb2ee4fc729c18d98444fed42fd0529eeeb` (2026). AI-generated; not peer reviewed.

[2] M. Petrache, S. Serfaty, *Crystallization for Coulomb and Riesz interactions as a consequence of the Cohn–Kumar conjecture*, arXiv:1908.09714v3 (2019).

[3] H. Cohn, A. Kumar, *Universally optimal distribution of points on spheres*, J. Amer. Math. Soc. **20** (2007), 99–148, arXiv:math/0607446.

[4] D. S. P. Salazar, *Bernstein functions at work: coalescents, copulas, and subordination*, arXiv:2607.04467v1 (2026); used only for the definitions (Def. 2.1, p. 4), for the statement of the Bernstein–Widder theorem (Thm 2.3, p. 5) and for the name "Bochner subordination" (p. 2).

[5] R. L. Schilling, R. Song, Z. Vondraček, *Bernstein Functions: Theory and Applications*, 2nd ed., de Gruyter, 2012. Existence of the monograph was confirmed through its publisher record and through [4]; its theorem on composition was *not* consulted or located.

[6] D. V. Widder, *The Laplace Transform*, Princeton University Press, 1941 (as cited in the Mathlib bibliography and in [3] p. 101); not consulted.

[7] Mathlib, `Mathlib/Analysis/Calculus/AbsolutelyMonotone.lean` (M. R. Douglas, 2025), at the pinned commit `d13f23b723b8a846827a245b89c10fc7d3f11612`.

[8] Lean community, *How to contribute*, section "Use of AI", leanprover-community.github.io/contribute/index.html, read 2026-10-25.

[9] X. Callens, *Riesz-energy, finiteness and every-density corollaries of the openai/math planar universal-optimality theorem, kernel-checked in Lean 4*, doi:10.5281/zenodo.23285631 (2026).

[10] X. Callens, *Planar universal optimality, tiered*, preprint v2, doi:10.5281/zenodo.23293429 (2026).

[11] X. Callens, *The Triangular Lattice and Universal Optimality* (textbook), doi:10.5281/zenodo.23295142 (2026).

[12] X. Callens, *The bilayer dipolar-exciton interaction is completely monotone in r^2: kernel-checked triangular-lattice corollaries at every density and an exact-Ewald CVODE consistency study*, doi:10.5281/zenodo.23646103 (2026).

[13] X. Callens, *The triangular lattice minimises the planar Yukawa energy at every density and for every finite mixture of screening lengths*, doi:10.5281/zenodo.23646105 (2026).

[14] X. Callens, *The Rytova–Keldysh interaction is a positive Gaussian mixture: kernel-checked admissibility for planar universal optimality and Keldysh-jellium Madelung energies of TMD Wigner lattices*, doi:10.5281/zenodo.23646107 (2026).

[15] X. Callens, *A cutoff-independent enstrophy bound for the $\sigma = 3$ dispersive dyadic shell model*, doi:10.5281/zenodo.23646111 (2026); sibling preprint of the same programme, not used mathematically here.

[16] X. Callens, *The Bogoliubov structure factor under the dual-scale envelope $\sqrt{k/2k_s}$*, doi:10.5281/zenodo.23646117 (2026); sibling preprint, not used mathematically here.